

Nombre:

DNI:

Fecha de nacimiento:

Paradigmas de la Programación

16/4/2026

Parcial 1

ej 1	ej 2	ej 3	ej 4	ej 5

1. En el siguiente programa, donde D1, D2, D3 son las **tres últimas cifras de tu DNI** (en ese orden), qué se imprime si el pasaje de parámetros es:
 - a. por valor:
 - b. por referencia:
 - c. por nombre:

None

```
procedure misterio(a, b)
begin
  a := a + 1;
  b := b + a;
end

x := D1;
y := D2;
z := D3;

misterio(x, y);
misterio(y, z);

print(x, y, z);
```

2. En el siguiente programa, donde D1 es el **día de su nacimiento** y D2 el **número del mes de su nacimiento**, qué se imprime si el alcance es:

a. estático:

b. dinámico:

None

```
x := D1;

procedure f() {
  print(x);
}

procedure g() {
  local x := D2;
  f();
}

procedure h() {
  x := D1 + D2;
  g();
}

h();
```

3. Dado el siguiente fragmento de código, considerado como una componente de software completa, indique si son verdaderas o falsas las siguientes afirmaciones:

JavaScript

```
let total = 0;
[1, 2, 3].forEach(n => {
  total += n;
});
```

V / F La variable `n` tiene efectos secundarios.

V / F La variable `total` tiene efectos secundarios.

V / F Hacemos asignación destructiva sobre la variable `n`.

V / F Hacemos asignación destructiva sobre la variable `total`.

V / F El código es determinístico e independiente del resto de la computación.

V / F El bloque **forEach** es independiente del resto de la computación.

4. Dadas estas dos variantes del mismo código:

Variante A:

None

```
case class User(id: Int, role: String)

object UserService {
  def getUser(id: Int): Option[User] = {
    Some(User(id, "user"))
  }
}

object AccessService {
  def checkAccess(user: User): Boolean = {
    user.role == "admin"
  }
}
```

Variante B:

None

```
object Processor {
  def process(handler: Any => Any, input: Any): Any = {
    handler(input)
  }
}

object UserService {
  def getUser(data: Any): Any = {
    val m = data.asInstanceOf[Map[String, Any]]
    if (m.contains("id")) {
      Map("id" -> m("id"), "role" -> "user")
    } else null
  }
}

object AccessService {
  def checkAccess(data: Any): Boolean = {
    val m = data.asInstanceOf[Map[String, Any]]
    m("role") == "admin"
  }
}
```

Indicar si son verdaderas o falsas las siguientes afirmaciones:

V / F La Variante A presenta más inseguridades de tipos que la variante B.

- V / F La Variante B es más flexible porque permite manejar cualquier tipo de entrada.
- V / F La Variante B es más segura porque permite manejar cualquier tipo de entrada.
- V / F La Variante A es más flexible porque permite manejar cualquier tipo de entrada.
- V / F La Variante A es más segura porque permite manejar cualquier tipo de entrada.
- V / F En la Variante B, el compilador puede detectar usos incorrectos de `User`.
- V / F La Variante A puede fallar incluso si compila correctamente.

5. Dado el siguiente programa:

Java

```
int compute(int a, int b) {  
    try {  
        int x = a / b;  
        if (x > 2) {  
            throw new RuntimeException("Grande");  
        }  
        return x;  
    } catch (ArithmeticException e) {  
        return -1;  
    } catch (RuntimeException e) {  
        return -2;  
    } finally {  
        System.out.println("Fin");  
    }  
}
```

Indicar el valor de retorno y qué se imprime para cada llamada:

1. si el día de tu cumpleaños es par: `compute(4, 2)`
2. si el día de tu cumpleaños es impar: `compute(5, 0)`